

# PHẦN 8.7

## GENERIC CLASS

*Từ Generic Method đến class tái sử dụng cho nhiều kiểu dữ liệu*

**Mục tiêu: hiểu vì sao cả class có thể dùng chung kiểu T, cách T được cố định khi tạo object và cách xây dựng GenericStorage<T>.**

Tài liệu học tập - Không kèm đáp án bài tập

## Phần 8.7 - Generic Class trong C#

Bạn đã học Generic Method:

```
public static T GetFirst<T>(List<T> items)
```

Đây là Generic Method vì chỉ method `GetFirst<T>()` sử dụng kiểu `T`.

Bây giờ ta học:

```
public class Storage<T>
{
}
```

Đây là Generic Class vì cả class `Storage<T>` có thể sử dụng kiểu `T`.

### 1. Generic Class giải quyết vấn đề gì?

Giả sử bạn cần class lưu trữ số nguyên:

```
public class IntStorage
{
    private readonly List<int> _items;

    public IntStorage()
    {
        _items = new List<int>();
    }

    public void Add(int item)
    {
        _items.Add(item);
    }

    public int GetFirst()
    {
        return _items[0];
    }
}
```

Sau đó bạn cần lưu chuỗi:

```
public class StringStorage
{
    private readonly List<string> _items;

    public StringStorage()
    {
        _items = new List<string>();
    }

    public void Add(string item)
    {
        _items.Add(item);
    }

    public string GetFirst()
    {
        return _items[0];
    }
}
```

```
}  
}
```

Rồi cần lưu Student:

```
public class StudentStorage  
{  
    private readonly List<Student> _items;  
  
    public StudentStorage()  
    {  
        _items = new List<Student>();  
    }  
  
    public void Add(Student item)  
    {  
        _items.Add(item);  
    }  
  
    public Student GetFirst()  
    {  
        return _items[0];  
    }  
}
```

Ba class có cùng logic:

- Tạo danh sách
- Thêm phần tử
- Lấy phần tử đầu
- Đếm phần tử
- Hiển thị phần tử

Chúng chỉ khác kiểu dữ liệu: int, string hoặc Student.

Ta thay cả ba class bằng một generic class:

```
public class Storage<T>  
{  
    private readonly List<T> _items;  
  
    public Storage()  
    {  
        _items = new List<T>();  
    }  
  
    public void Add(T item)  
    {  
        _items.Add(item);  
    }  
  
    public T GetFirst()  
    {  
        return _items[0];  
    }  
}
```

Bây giờ có thể tạo:

```
Storage<int> numberStorage = new Storage<int>();
```

```
Storage<string> nameStorage = new Storage<string>();
Storage<Student> studentStorage = new Storage<Student>();
```

**Generic Class cho phép viết một class một lần, rồi sử dụng class đó với nhiều kiểu dữ liệu khác nhau.**

## 2. Cú pháp Generic Class

```
public class ClassName<T>
{
}
```

Ví dụ:

```
public class Box<T>
{
    private T _value;
}
```

| Thành phần | Ý nghĩa                   |
|------------|---------------------------|
| Box        | Tên class                 |
| <T>        | Khai báo type parameter T |
| T _value   | Field có kiểu T           |

Khi tạo object:

```
Box<int> numberBox = new Box<int>();
```

thì T = int. Khi tạo:

```
Box<string> textBox = new Box<string>();
```

thì T = string.

## 3. T thuộc về toàn bộ class

```
public class Box<T>
{
    private T _value;

    public void SetValue(T value)
    {
        _value = value;
    }

    public T GetValue()
    {
        return _value;
    }
}
```

T có thể được sử dụng ở:

- field
- property
- parameter
- return type
- collection
- method bên trong class

Trong class trên, field, parameter và return type đều dùng cùng một T được xác định khi object được tạo.

## 4. Khi nào kiểu T được xác định?

```
public class Box<T>
{
    private T _value;

    public Box(T value)
    {
        _value = value;
    }

    public T GetValue()
    {
        return _value;
    }
}
```

Khi tạo:

```
Box<int> numberBox = new Box<int>(100);
```

thì toàn bộ T trong object đó trở thành int. Bạn có thể hình dung class như sau:

```
public class Box<int>
{
    private int _value;

    public Box(int value)
    {
        _value = value;
    }

    public int GetValue()
    {
        return _value;
    }
}
```

Đây chỉ là cách hình dung để dễ hiểu. Với Box<string>, toàn bộ class hoạt động với string.

## 5. Một object Generic Class chỉ dùng một kiểu đã chọn

```
Box<int> numberBox = new Box<int>(10);
```

Object này chỉ làm việc với int.

Đúng:

```
numberBox.SetValue(20);
```

Sai:

```
numberBox.SetValue("Hello");
```

Vì khi tạo `Box<int>`, bạn đã cố định `T = int`. Tương tự, `Box<Student>` chỉ nhận `Student`.

## 6. Generic Class khác Generic Method thế nào?

### Generic Method

```
public static T GetFirst<T>(List<T> items)
{
    return items[0];
}
```

`T` chỉ thuộc về method này. Mỗi lần gọi, `T` có thể khác nhau:

```
GetFirst(numbers); // T = int
GetFirst(names);   // T = string
GetFirst(students); // T = Student
```

### Generic Class

```
public class Storage<T>
{
}
```

`T` thuộc về toàn bộ class. Khi tạo `Storage<int>`, object đó dùng `int` trong tất cả method của nó.

**`MethodName<T>()` là Generic Method. `ClassName<T>` là Generic Class.**

## 7. Ví dụ cơ bản - Box<T>

```
public class Box<T>
{
    private T _value;

    public Box(T value)
    {
        _value = value;
    }

    public void SetValue(T value)
    {
        _value = value;
    }

    public T GetValue()
    {
        return _value;
    }
}
```

```

    public void DisplayValue()
    {
        Console.WriteLine(
            $"Type: {typeof(T).Name}, Value: {_value}");
    }
}

```

### Sử dụng với int

```

Box<int> numberBox =
    new Box<int>(100);

numberBox.DisplayValue();
numberBox.SetValue(200);
int number = numberBox.GetValue();

```

Kết quả: Type: Int32, Value: 100.

### Sử dụng với string

```

Box<string> textBox =
    new Box<string>("C#");

textBox.DisplayValue();
textBox.SetValue(".NET");
string text = textBox.GetValue();

```

### Sử dụng với object

```

Student student =
    new Student("SV01", "Hung");

Box<Student> studentBox =
    new Box<Student>(student);

Student storedStudent =
    studentBox.GetValue();

```

Không cần ép kiểu vì compiler biết Box<Student> làm cho GetValue() trả về Student.

## 8. Generic Class chứa List<T>

```

public class Storage<T>
{
    private readonly List<T> _items;

    public Storage()
    {
        _items = new List<T>();
    }
}

```

Ở đây có hai generic class cùng xuất hiện: Storage<T> do chúng ta viết và List<T> do .NET cung cấp. Cả hai dùng chung kiểu T.

Ví dụ, khi tạo `Storage<Student>`, `List<T>` bên trong trở thành `List<Student>`.

## 9. Ví dụ đầy đủ - `Storage<T>`

```
public class Storage<T>
{
    public string Name { get; }

    private readonly List<T> _items;

    public Storage(string name)
    {
        if (string.IsNullOrEmpty(name))
        {
            throw new ArgumentException(
                "Name cannot be null or whitespace.",
                nameof(name));
        }

        Name = name.Trim();
        _items = new List<T>();
    }

    public void Add(T item)
    {
        _items.Add(item);
    }

    public T GetFirst()
    {
        if (_items.Count == 0)
        {
            throw new InvalidOperationException(
                "The storage is empty.");
        }

        return _items[0];
    }

    public T GetLast()
    {
        if (_items.Count == 0)
        {
            throw new InvalidOperationException(
                "The storage is empty.");
        }

        return _items[_items.Count - 1];
    }

    public T GetByIndex(int index)
    {
        if (index < 0 || index >= _items.Count)
        {
            throw new ArgumentOutOfRangeException(
                nameof(index),
                index,
                "Index is outside the storage range.");
        }

        return _items[index];
    }
}
```



```

    }

    public int GetCount()
    {
        return _items.Count;
    }

    public void DisplayAll()
    {
        Console.WriteLine($"Storage: {Name}");

        if (_items.Count == 0)
        {
            Console.WriteLine(
                "The storage is empty.");
            return;
        }

        foreach (T item in _items)
        {
            Console.WriteLine(item);
        }
    }
}

```

## 10. Sử dụng Storage<int>

```

Storage<int> numberStorage =
    new Storage<int>("Numbers");

numberStorage.Add(10);
numberStorage.Add(20);
numberStorage.Add(30);

numberStorage.DisplayAll();

int firstNumber =
    numberStorage.GetFirst();

int lastNumber =
    numberStorage.GetLast();

```

Ở object này, T = int. Vì vậy Add(T item) được hiểu là Add(int item), còn GetFirst() trả về int.

## 11. Sử dụng Storage<string>

```

Storage<string> nameStorage =
    new Storage<string>("Names");

nameStorage.Add("Hung");
nameStorage.Add("An");
nameStorage.Add("Binh");

string firstName =
    nameStorage.GetFirst();

nameStorage.DisplayAll();

```

Ở đây T = string. Object này không thể thêm số nguyên.

## 12. Sử dụng Storage<Student>

```
public class Student
{
    public string Id { get; }
    public string FullName { get; }

    public Student(
        string id,
        string fullName)
    {
        if (string.IsNullOrEmpty(id))
        {
            throw new ArgumentException(
                "Id cannot be null or whitespace.",
                nameof(id));
        }

        if (string.IsNullOrEmpty(fullName))
        {
            throw new ArgumentException(
                "Full name cannot be null or whitespace.",
                nameof(fullName));
        }

        Id = id.Trim();
        FullName = fullName.Trim();
    }

    public override string ToString()
    {
        return $"Id: {Id}, Full name: {FullName}";
    }
}
```

```
Storage<Student> studentStorage =
    new Storage<Student>("Students");

studentStorage.Add(
    new Student("SV01", "Hung"));

studentStorage.Add(
    new Student("SV02", "An"));

studentStorage.DisplayAll();
```

Do Student đã override ToString(), Console.WriteLine(item) sẽ hiển thị thông tin có ý nghĩa. Nếu không override ToString(), kết quả có thể chỉ giống Namespace.Student.

## 13. Vì sao DisplayAll() không gọi DisplayInfo()?

Bạn có thể muốn viết:

```
foreach (T item in _items)
{
    item.DisplayInfo();
}
```

Nhưng compiler không cho phép vì T có thể là int, string, Student hoặc Product. Không phải mọi kiểu đều có DisplayInfo().

Với Generic không có constraint, compiler chỉ biết T là một kiểu bất kỳ. Vì vậy cách chung là:

```
Console.WriteLine(item);
```

Sau này ở Generic Constraints, ta có thể yêu cầu where T : IDisplayable. Khi đó compiler mới chắc rằng T có method cần thiết.

## 14. Generic Class có nhiều type parameter

```
public class Pair<TKey, TValue>
{
    public TKey Key { get; }
    public TValue Value { get; }

    public Pair(
        TKey key,
        TValue value)
    {
        Key = key;
        Value = value;
    }

    public void Display()
    {
        Console.WriteLine(
            $"Key: {Key}, Value: {Value}");
    }
}
```

Ví dụ:

```
Pair<string, string> studentPair =
    new Pair<string, string>(
        "SV01",
        "Hung");
```

TKey = string, TValue = string.

```
Pair<string, decimal> productPair =
    new Pair<string, decimal>(
        "SP01",
        1_000_000m);
```

TKey = string, TValue = decimal.

```
Pair<int, Student> result =
    new Pair<int, Student>(
        1,
        student);
```

TKey = int, TValue = Student.

## 15. Generic Class có thể chứa Generic Method

```
public class Storage<T>
{
    private readonly List<T> _items =
        new List<T>();

    public void Add(T item)
    {
        _items.Add(item);
    }

    public void DisplayOther<TAnother>(
        TAnother value)
    {
        Console.WriteLine(value);
    }
}
```

T là type parameter của class, còn TAnother là type parameter riêng của method. Hai type parameter này độc lập.

```
Storage<int> storage =
    new Storage<int>();

storage.Add(10);           // T = int
storage.DisplayOther("Hello"); // TAnother = string
```

## 16. Constructor của Generic Class

Constructor không viết lại <T>.

Đúng:

```
public class Box<T>
{
    public Box(T value)
    {
    }
}
```

Sai:

```
public Box<T>(T value)
```

Constructor đã thuộc class Box<T>, nên nó đã biết T.

## 17. Có được tạo new T() trong Generic Class không?

Thông thường, chưa thể viết:

```
public class Factory<T>
{
    public T Create()
    {
        return new T();
    }
}
```

```
}  
}
```

Compiler không chắc T có constructor không tham số. Muốn làm vậy cần:

```
public class Factory<T>  
    where T : new()  
{  
    public T Create()  
    {  
        return new T();  
    }  
}
```

where T : new() sẽ được học kỹ trong Phần 8.8 - Generic Constraints.

## 18. Có thể dùng toán tử với T không?

```
public class Calculator<T>  
{  
    public T Add(T first, T second)  
    {  
        return first + second;  
    }  
}
```

Cách này không đơn giản được chấp nhận với một T bất kỳ. Compiler không biết T có hỗ trợ + hay kết quả có phải là T không. T có thể là int, decimal, string, Student hoặc Order.

## 19. So sánh hai phần tử kiểu T

Có thể dùng:

```
EqualityComparer<T>.Default.Equals(  
    first,  
    second);
```

Ví dụ:

```
public bool Contains(T value)  
{  
    foreach (T item in _items)  
    {  
        if (EqualityComparer<T>  
            .Default  
            .Equals(item, value))  
        {  
            return true;  
        }  
    }  
    return false;  
}
```

Hoặc đơn giản hơn, vì List<T> đã hỗ trợ Contains():

```
public bool Contains(T value)
{
    return _items.Contains(value);
}
```

List<T>.Contains() cũng sử dụng cơ chế equality phù hợp cho T.

## 20. Xóa phần tử kiểu T

```
public bool Remove(T item)
{
    return _items.Remove(item);
}
```

| Kết quả | Ý nghĩa                 |
|---------|-------------------------|
| true    | Tìm thấy và xóa phần tử |
| false   | Không tìm thấy phần tử  |

Method này dùng được với Storage<int>, Storage<string> và Storage<Student>. Với object như Student, kết quả phụ thuộc vào cách equality được định nghĩa.

## 21. Kiểu Generic được giữ an toàn

```
Storage<Student> storage =
    new Storage<Student>("Students");
```

Compiler cho phép:

```
storage.Add(student);
```

Nhưng không cho phép:

```
storage.Add("Hello");
```

Kết quả lấy ra không cần cast:

```
Student student = storage.GetFirst();
```

**Lợi ích chính của Generic Class: tái sử dụng code + an toàn kiểu + không cần ép kiểu.**

## 22. Generic Class trong .NET

| Generic Class            | Ví dụ sử dụng               | Kiểu được thay                  |
|--------------------------|-----------------------------|---------------------------------|
| List<T>                  | List<Student>               | T = Student                     |
| Queue<T>                 | Queue<Order>                | T = Order                       |
| Stack<T>                 | Stack<string>               | T = string                      |
| HashSet<T>               | HashSet<int>                | T = int                         |
| Dictionary<TKey, TValue> | Dictionary<string, Student> | TKey = string, TValue = Student |
| Task<T>                  | Task<Student>               | T = Student                     |
| DbSet<T>                 | DbSet<Product>              | T = Product                     |

.NET viết các class Generic một lần và cho phép lập trình viên sử dụng với nhiều kiểu.

## 23. Ví dụ trong ASP.NET Core sau này

```
public class Repository<T>
{
    private readonly List<T> _items;

    public Repository()
    {
        _items = new List<T>();
    }

    public void Add(T entity)
    {
        _items.Add(entity);
    }

    public List<T> GetAll()
    {
        return new List<T>(_items);
    }
}
```

```
Repository<Student> studentRepository =
    new Repository<Student>();

Repository<Product> productRepository =
    new Repository<Product>();
```

Một class quản lý nhiều loại entity. Trong dự án thật, repository thường kết hợp với interface, Entity Framework Core, constraints và async/await; nền tảng chính vẫn là Generic Class.

## 24. Không nên trả trực tiếp collection nội bộ

Không nên:

```
public List<T> Items => _items;
```

Vì bên ngoài có thể Clear(), Add() hoặc Remove() trực tiếp, làm class mất quyền kiểm soát.

Có thể trả về bản sao:

```
public List<T> GetAll()
{
    return new List<T>(_items);
}
```

Hoặc sau này dùng IReadOnlyList<T>:

```
public IReadOnlyList<T> GetAll()
{
    return _items.AsReadOnly();
}
```

Ở giai đoạn hiện tại, trả về bản sao là cách dễ hiểu nhất.

## 25. Những lỗi thường gặp

### Lỗi 1: Quên <T> sau tên class

```
// Sai
public class Storage
{
    private List<T> _items;
}

// Đúng
public class Storage<T>
{
    private List<T> _items;
}
```

### Lỗi 2: Viết <T> trên constructor

```
// Sai
public Storage<T>()
{
}

// Đúng
public Storage()
{
}
```

### Lỗi 3: Tạo object nhưng không truyền kiểu

```
// Không đầy đủ
Storage storage = new Storage();

// Đúng
Storage<int> storage =
    new Storage<int>("Numbers");
```

Generic Class thường cần ghi rõ kiểu khi tạo object, khác với nhiều Generic Method có thể suy luận kiểu từ argument.

### Lỗi 4: Trộn kiểu trong cùng một object

```
Storage<int> storage =
    new Storage<int>("Numbers");

storage.Add(10);
storage.Add(20);
// storage.Add("Hello"); // Sai
```

### Lỗi 5: Gọi method không được bảo đảm trên T

```
item.DisplayInfo();
```



Compiler không biết mọi T đều có DisplayInfo().

### Lỗi 6: Dùng phép toán trực tiếp trên T

```
first + second
first > second
```

Compiler chưa biết khả năng của T.

### Lỗi 7: Trả collection nội bộ ra ngoài

```
public List<T> Items => _items;
```

Điều này làm hỏng tính đóng gói.

## 26. Tóm tắt Generic Method và Generic Class

| Nội dung               | Generic Method | Generic Class  |
|------------------------|----------------|----------------|
| Cú pháp                | Method<T>()    | Class<T>       |
| Phạm vi của T          | Trong method   | Toàn bộ class  |
| Xác định T             | Khi gọi method | Khi tạo object |
| Ví dụ                  | GetFirst<T>()  | Storage<T>     |
| Có thể dùng nhiều kiểu | Có             | Có             |

Ví dụ Generic Method:

```
public static T GetFirst<T>(
    List<T> items)
{
    return items[0];
}
```

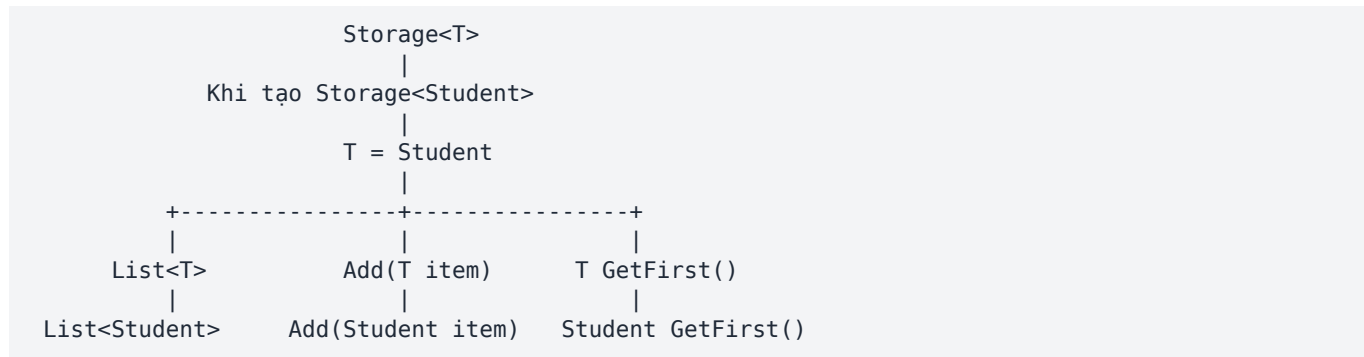
Ví dụ Generic Class:

```
public class Storage<T>
{
    private readonly List<T> _items;
}
```

## 27. Cách đọc Generic Class

| Đoạn code                        | Cách đọc bằng tiếng Việt                                                 |
|----------------------------------|--------------------------------------------------------------------------|
| public class Storage<T>          | Khai báo một class Storage làm việc với một kiểu chưa xác định tên là T. |
| private readonly List<T> _items; | Class có một danh sách chứa các phần tử kiểu T.                          |
| public void Add(T item)          | Method nhận một phần tử đúng kiểu T của Storage.                         |
| public T GetFirst()              | Method trả về một phần tử đúng kiểu T.                                   |
| Storage<Student>                 | Một Storage mà kiểu phần tử được xác định là Student.                    |

## 28. Sơ đồ tư duy



## 29. Điều cần nhớ nhất

**Generic Method: một method dùng được với nhiều kiểu. Generic Class: một class dùng được với nhiều kiểu.**

T là kiểu đại diện, nhưng khi tạo object, T được cố định cho object đó:

- Storage<int> chỉ làm việc với int.
- Storage<string> chỉ làm việc với string.
- Storage<Student> chỉ làm việc với Student.

## Bài tập 8.7 - GenericStorage<T>

Tạo class:

```
public class GenericStorage<T>
```

### Thuộc tính và collection

```
public string Name { get; }
private readonly List<T> _items;
```

### Constructor

```
public GenericStorage(string name)
```

Yêu cầu:

- name không được rỗng;
- dùng Trim();
- khởi tạo \_items.

### Method 1

```
public void AddItem(T item)
```

Yêu cầu:

- thêm item vào `_items`.

## Method 2

```
public T GetFirstItem()
```

Yêu cầu:

- nếu storage rỗng, ném `InvalidOperationException`;
- trả về phần tử đầu tiên.

## Method 3

```
public T GetLastItem()
```

Yêu cầu:

- nếu storage rỗng, ném `InvalidOperationException`;
- trả về phần tử cuối cùng.

## Method 4

```
public T GetItemAt(int index)
```

Yêu cầu:

- kiểm tra index hợp lệ;
- nếu không hợp lệ, ném `ArgumentOutOfRangeException`;
- trả về phần tử tại index.

## Method 5

```
public bool RemoveItem(T item)
```

Yêu cầu:

- dùng `List<T>.Remove()`;
- trả về true hoặc false.

## Method 6

```
public bool ContainsItem(T item)
```

Yêu cầu:

- dùng `List<T>.Contains()`.

## Method 7

```
public int GetItemCount()
```

Yêu cầu:

- trả về Count.

## Method 8

```
public List<T> GetAllItems()
```

Yêu cầu:

- không trả trực tiếp \_items;
- trả về một danh sách mới chứa dữ liệu hiện tại.

Gợi ý:

```
return new List<T>(_items);
```

## Method 9

```
public void DisplayItems()
```

Yêu cầu:

- hiển thị Name;
- kiểm tra storage rỗng;
- dùng foreach;
- không thay đổi collection.

## Yêu cầu trong Program.cs

Tạo ba storage khác nhau:

```
GenericStorage<int>  
GenericStorage<string>  
GenericStorage<Product>
```

### Với GenericStorage<int>

1. Thêm ít nhất ba số.
2. Lấy phần tử đầu.
3. Lấy phần tử cuối.
4. Kiểm tra một số có tồn tại.
5. Xóa một số.
6. Hiển thị toàn bộ.

### Với GenericStorage<string>

7. Thêm ba tên.
8. Lấy một phần tử theo index.
9. Xóa một tên.
10. Hiển thị danh sách.

### Với GenericStorage<Product>

11. Tạo ít nhất hai sản phẩm.
12. Thêm vào storage.
13. Hiển thị.
14. Lấy sản phẩm đầu tiên.
15. Kiểm tra GetItemCount().

### Kiểm thử lỗi

- Lấy phần tử từ storage rỗng.
- Truy cập index không hợp lệ.
- Xử lý bằng try-catch.

**Trước khi sang Phần 8.8 - Generic Constraints, nên hoàn thành cả bài GenericHelper và GenericStorage<T> vì Constraints sử dụng trực tiếp hai nền tảng này.**

### Checklist trước khi nộp bài

- ☐ Class được khai báo đúng là GenericStorage<T>.
- ☐ Collection là private readonly List<T>.
- ☐ Constructor validation name trước khi Trim().
- ☐ GetFirstItem() và GetLastItem() kiểm tra storage rỗng.
- ☐ GetItemAt() kiểm tra cả index âm và index vượt Count - 1.
- ☐ RemoveItem() và ContainsItem() trả về bool đúng nghĩa.
- ☐ GetAllItems() trả về bản sao, không trả trực tiếp \_items.
- ☐ DisplayItems() không thay đổi collection trong foreach.
- ☐ Program kiểm thử với int, string và Product.
- ☐ Có kiểm thử exception bằng try-catch.